

PROJECT 2 – Password Security & Cracking Analysis

This report summarizes the controlled password security analysis performed.

Verified tools

Created passwords.txt

Generated hashes

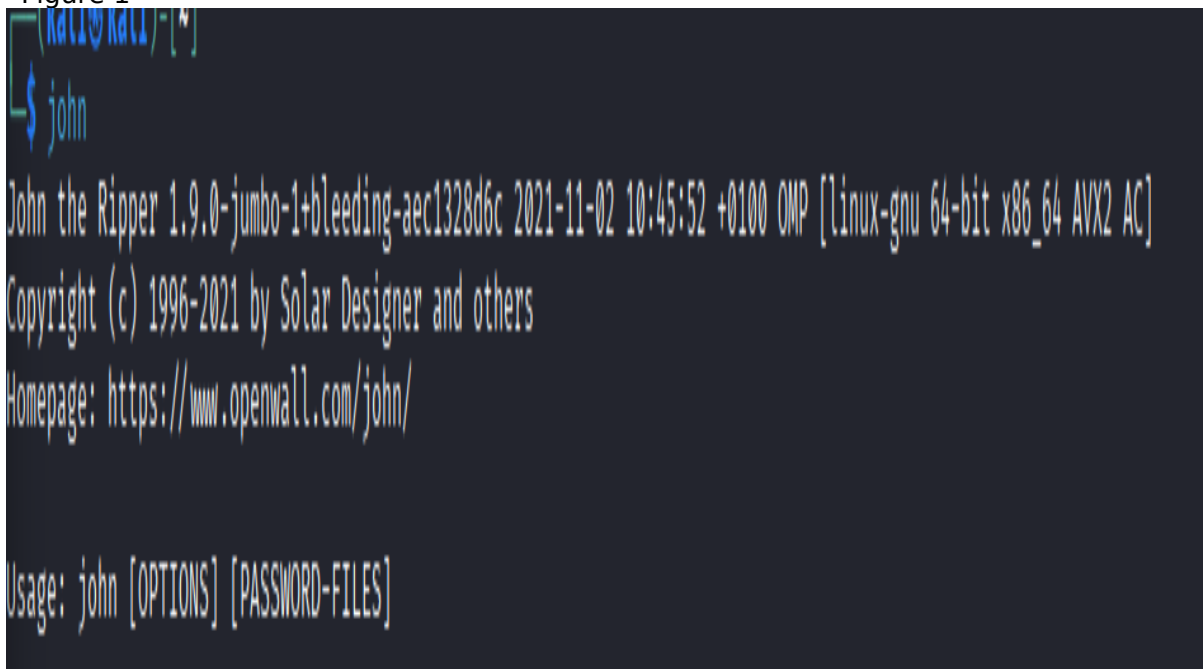
Identified hash type

Generated wordlist

Analyzed hashes with John

Result: 4 password hashes cracked, 0 left

Figure 1



```
(kali@kali)~$ john
John the Ripper 1.9.0-jumbo-1+bleeding-aec1328d6c 2021-11-02 10:45:52 +0100 OMP [linux-gnu 64-bit x86_64 AVX2 AC]
Copyright (c) 1996-2021 by Solar Designer and others
Homepage: https://www.openwall.com/john/

Usage: john [OPTIONS] [PASSWORD-FILES]
```

Figure 2

```
→$ crunch
crunch version 3.6

Crunch can create a wordlist based on criteria you specify. The output from crunch can be sent to the screen, file
, or to another program.

Usage: crunch <min> <max> [options]
where min and max are numbers

Please refer to the man page for instructions and examples on how to use crunch.
```

Figure 3

```
(kali@kali)-[~]
└─$ openssl version
OpenSSL 3.6.2 7 Apr 2026 (Library: OpenSSL 3.6.2 7 Apr 2026)
```

Figure 4

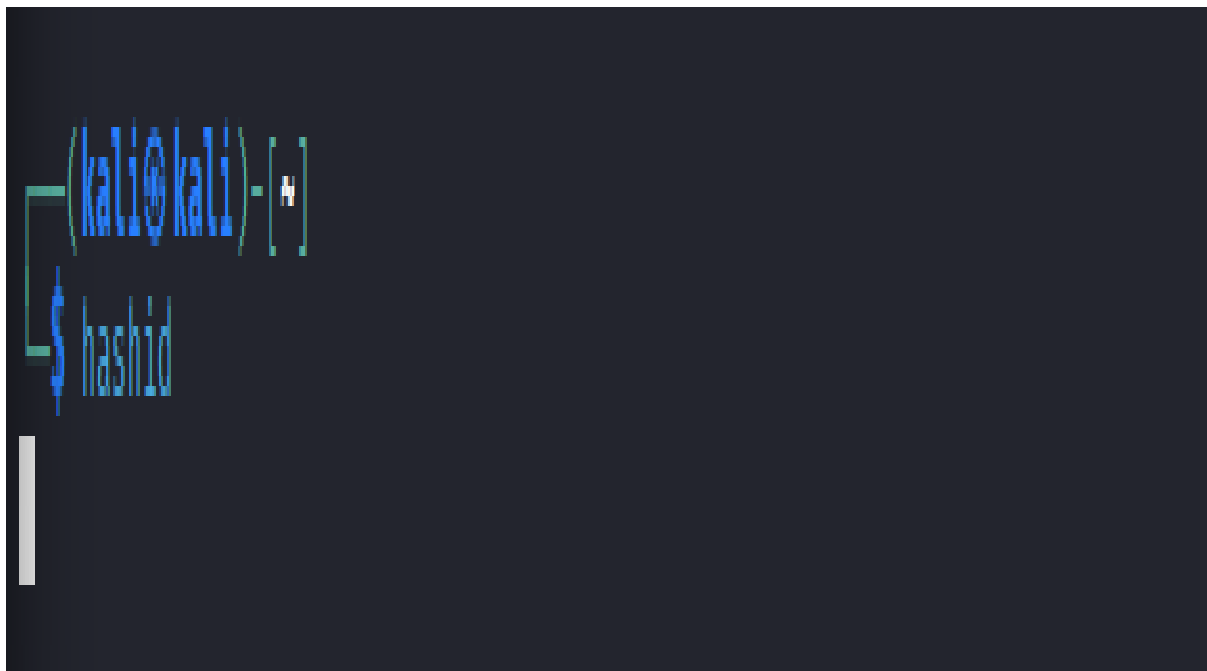


Figure 5

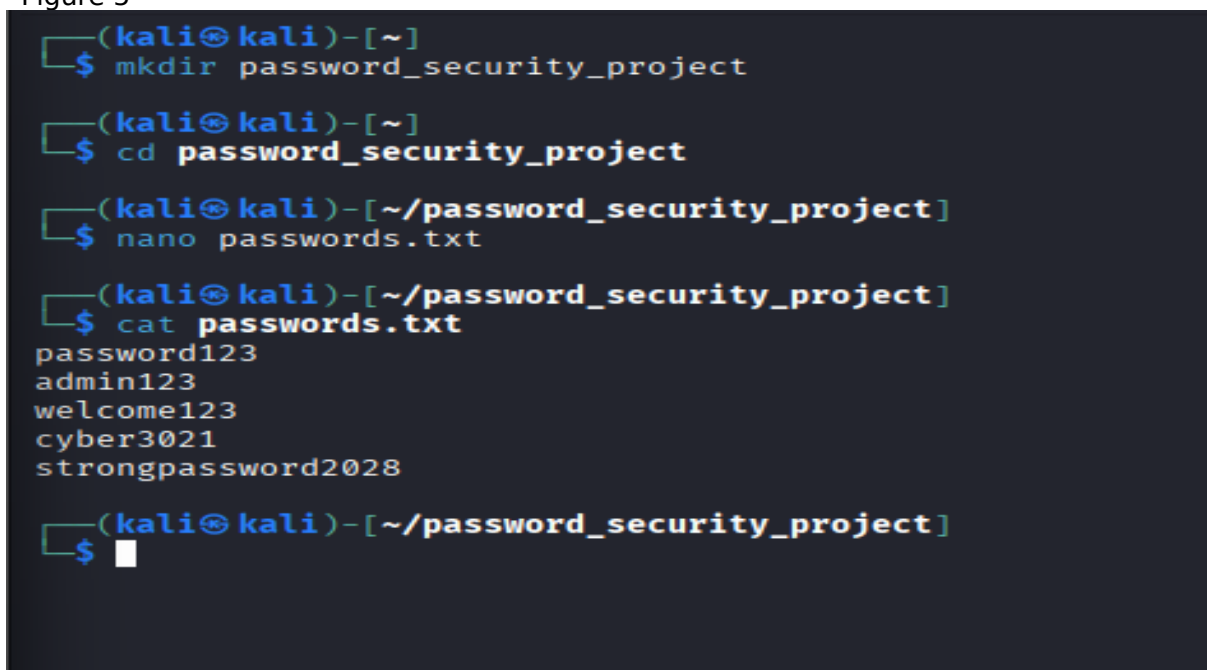


Figure 6

```
(kali@kali)-[~/password_security_project]
$ echo -n "password123" | openssl dgst -sha256 >> hashes.txt
```

Figure 7

```
(kali@kali)-[~/password_security_project]
$ echo -n "password123" | openssl dgst -sha256 >> hashes.txt

(kali@kali)-[~/password_security_project]
$ cat hashes.txt
SHA2-256(stdin)= ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
```

Figure 8

```

(kali@kali)-[~/password_security_project]
$ touch hashes.txt

(kali@kali)-[~/password_security_project]
$ echo -n "password123" | openssl dgst -sha256 >> hashes.txt

(kali@kali)-[~/password_security_project]
$ cat hashes.txt
SHA2-256(stdin)= ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f

(kali@kali)-[~/password_security_project]
$ echo -n "welcome123" | openssl dgst -sha256 >> hashes.txt

(kali@kali)-[~/password_security_project]
$ echo -n "cyber3021" | openssl dgst -sha256 >> hashes.txt

(kali@kali)-[~/password_security_project]
$ echo -n "strongpassword2028" | openssl dgst -sha256 >> hashes.txt

(kali@kali)-[~/password_security_project]
$ cat hashes.txt
SHA2-256(stdin)= ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
SHA2-256(stdin)= a68349561396ec264a350847024a4521d00beaa3358660c2709a80f31c7acdd0
SHA2-256(stdin)= 58f1e5c5cbd9d501c7e879e834764c8a96b3320d4349d71f97a5205cb95d8741
SHA2-256(stdin)= 651298d37915a8029a9fd3a38dce07d5840b2794f42426615eee30a13527b219

(kali@kali)-[~/password_security_project]
$ █

```

Figure 9

```

(kali@kali)-[~/password_security_project]
$ ls
hashes.txt  passwords.txt

(kali@kali)-[~/password_security_project]
$ █

```

Conclusion: Strong unique passwords improve security.